

Deltx: An Intelligent Static Analysis and Predictive Analytics Platform for Software Quality Evolution

Interim Research Report

S.M. Silva	Index No: 22001905
Praneesh S.	Index No: 22001591
R.M.I.M. De Silva	Index No: 22000331

Supervisor

Dr. Manjusri Wickramasinghe

University of Colombo School of Computing
Colombo, Sri Lanka

October 2026

List of Acronyms

AI	Artificial Intelligence
AST	Abstract Syntax Tree
CI/CD	Continuous Integration and Continuous Deployment
ECDF	Empirical Cumulative Distribution Function
GRU	Gated Recurrent Unit
LLM	Large Language Model
LSTM	Long Short-Term Memory
LTSF	Long-Term Time Series Forecasting
MAE	Mean Absolute Error
MLP	Multilayer Perceptron
MQR	Multi-Quality Rule mode in SonarQube
NCLOC	Non-comment Lines of Code
PatchTST	Patch Time Series Transformer
RMSE	Root Mean Square Error
SHAP	SHapley Additive exPlanations
SQUALE	Software QUALity Enhancement
XAI	Explainable Artificial Intelligence

Contents

List of Acronyms	i
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement and Research Gaps	1
1.2.1 Issue 1: Invisibility of AI-Generated Structural Decay	1
1.2.2 Issue 2: Semantic Gap in Software Quality Metrics	2
1.2.3 Issue 3: Retrospective Nature of Static Analysis	2
1.2.4 Issue 4: Lack of Interpretability in Predictive Metrics	2
1.3 Scope	2
1.4 Research Aim	3
1.5 Current Progress	3
2 Literature Review	5
2.1 AI-Generated Text and Code Detection	5
2.2 Hierarchical Software Quality Models	6
2.3 Predictive Time-Series Modeling	6
2.4 Explainable Artificial Intelligence	6
3 Research Objectives / Questions	7
3.1 Research Objectives	7
4 Research Approach and Methodology	9
4.1 System Architecture Overview	9
4.2 Dataset Construction and Checkpoint Selection	9
4.3 AI Authorship Detection Methodology	10
4.4 Repository Quality Assessment	11

4.5	Quality Mapping	11
4.6	Dynamic Issue Weighting	12
4.7	Individual Marks and SQUALE Aggregation	12
4.8	Maintainability Composite	13
4.9	Dataset Vector	13
4.10	Predictive Modeling and Interpretation	15
5	Experiments and Preliminary Results	16
5.1	Repositories Analysed	16
5.1.1	Dataset Extraction Runtime and Compute Cost	17
5.2	Repository-Level Exploratory Analysis	18
5.3	Forecasting Experiments	18
5.3.1	Rolling-Origin Evaluation Protocol	18
5.3.2	Preliminary ARIMA Results for Maintainability	19
5.4	Preliminary Findings	20
6	Finalized Evaluation Plan	21
6.1	AI Code Detection Evaluation	21
6.2	Dataset Integrity and Reproducibility	21
6.3	Scoring Validation	22
6.4	Predictive Modeling Accuracy	22
6.5	Explainability and Practical Usefulness	23
A	Repository-Level Exploratory Visualizations	24
B	ARIMA Maintainability Forecast Outputs	37
	References	44

Chapter 1

Introduction

1.1 Background and Motivation

The software engineering landscape is undergoing a major shift driven by the adoption of Large Language Models (LLMs) and AI-assisted programming agents. AI-assisted development can accelerate implementation, but it also changes how software is produced and can introduce structural vulnerabilities, architectural inconsistencies, latent technical debt, and other risks that may only become visible as the codebase evolves. Existing empirical work has reported measurable differences between human-written and AI-generated code in defects, vulnerabilities, and complexity [1]. At the same time, AI involvement is often not explicitly declared in real repositories, which makes it difficult to study its longitudinal effect on software quality [2].

Software quality is described at a high level by standards such as ISO/IEC 25010, while static-analysis platforms such as SonarQube provide operational measurements and rule violations. However, static analysis is fundamentally retrospective: it reports the current condition of a codebase rather than predicting how software quality may evolve in future commits.

1.2 Problem Statement and Research Gaps

1.2.1 Issue 1: Invisibility of AI-Generated Structural Decay

Current static-analysis tools process source code without distinguishing human-authored code from AI-generated code. AI-generated code can be functionally correct in isolation while introducing structural problems when integrated into a larger codebase. Without an authorship signal, organizations cannot separately observe the evolution of AI-assisted code or study whether changes in AI-authorship are associated

with later quality degradation.

1.2.2 Issue 2: Semantic Gap in Software Quality Metrics

SonarQube produces granular rule violations and structural measurements, but there is a semantic gap between these low-level outputs and higher-level software-quality dimensions such as Correctness, Maintainability, Security, and Efficiency. Simple counting or arithmetic aggregation can also mask severe problems when many less important observations are present. SQUALE-style nonlinear aggregation is used in DELTX to address this conflict-resistance requirement [3, 4].

1.2.3 Issue 3: Retrospective Nature of Static Analysis

Static-analysis platforms provide point-in-time snapshots but do not directly forecast how quality will evolve across future commits. This leaves software teams in a reactive position, where degradation is addressed only after it has already appeared. DELTX addresses this by constructing chronological repository-quality histories that can be used for multivariate time-series forecasting.

1.2.4 Issue 4: Lack of Interpretability in Predictive Metrics

Predictive models can be difficult for developers to trust when they provide only numerical forecasts. If the system predicts a future decline in Maintainability, Correctness, Security, or Efficiency, developers need an explanation of which historical variables and time periods influenced that forecast. The planned interpretation layer therefore uses SHAP-based feature attribution [5].

1.3 Scope

The project scope is intentionally restricted to Python repositories. The current research pipeline consists of five stages: repository-history extraction, AI-authorship detection, SQUALE-based quality aggregation, multivariate time-series forecasting, and SHAP-based interpretation. The first three stages are implemented. Forecasting experiments have begun using datasets extracted from multiple repositories, while the final interpretation layer remains planned.

Commits that contain no meaningful Python syntax change are excluded using AST-based semantic comparison; documentation-only, comment-only, docstring-only, whitespace-only, formatting-only, and equivalent-rename commits are filtered.

1.4 Research Aim

The research aim is to conceptualize, design, and evaluate DELTX, an architecture integrating static code analysis, AI detection, predictive modeling, and explainability in order to transform retrospective software-quality measurements into proactive quality trajectories.

1.5 Current Progress

Table 1.1 summarizes the interim implementation state.

Table 1.1: Current implementation progress.

Stage	Module	Status	Current state
1	<code>deltx.extraction</code>	Implemented	Python history extraction, semantic checkpoint filtering, churn, topology, dataset orchestration.
2	<code>deltx.detection</code>	Implemented	DroidDetect-Base-Binary inference with file/chunk aggregation to <code>ai_confidence_pct</code> .
3	<code>deltx.scoring</code>	Implemented	Dockerized SonarQube analysis, modern MQR mappings, dynamic issue weighting, SQUALE-based four-score aggregation.
4	<code>deltx.prediction</code>	Experimental	Multiple forecasting models have been trained; final comparative results are to be inserted from the experiment outputs.
5	<code>deltx.interpretation</code>	Planned	SHAP-based explanation of the selected forecasting model.

Chapter 2

Literature Review

The theoretical foundation is organized around four domains: AI-generated code detection, hierarchical software-quality modeling, predictive time-series modeling, and Explainable Artificial Intelligence.

2.1 AI-Generated Text and Code Detection

The growing use of LLMs has motivated methods for distinguishing synthetic content from human authorship. Earlier approaches often depended on perplexity, token likelihoods, or related statistical signatures.

The project uses the published DroidDetect-Base-Binary checkpoint from the Droid resource suite [6]. The detector is fine-tuned from ModernBERT-base and covers Python among its supported languages. The binary head separates human-generated and machine-generated code, where the machine-generated class includes generated, refined, and adversarial examples. DELTX does not train a new detector; it uses the published detector as an authorship signal and aggregates file-level outputs into a commit-level score.

The current score is

$$\text{ai_confidence_pct} = 100 (1 - P(\text{HUMAN_GENERATED})) . \quad (2.1)$$

The detector output is treated as an ordinal confidence signal rather than a calibrated probability. The implementation also preserves full-file context where possible because fragment-level evaluation was observed to produce substantially higher false-positive behavior in local verification.

2.2 Hierarchical Software Quality Models

ISO/IEC 25010 provides a high-level vocabulary for software quality, but operational tools produce low-level measurements and rule violations. Bridging this abstraction gap is a long-standing software-engineering problem. The Quamoco approach formalized mappings between concrete measurements and quality factors [7]. SQUALE introduced a practice-based quality model that normalizes heterogeneous observations into marks and aggregates them using nonlinear functions that give greater influence to poor values [3, 4].

These quality models inform the scoring methodology. DELTX uses SonarQube as the measurement instrument and SQUALE-inspired nonlinear aggregation as the mechanism for converting granular findings into four continuous target scores. Current SonarQube MQR findings can carry separate Maintainability, Reliability, and Security impacts, and the implemented scoring model preserves those multi-impact semantics rather than collapsing an issue to a single legacy type.

2.3 Predictive Time-Series Modeling

Historical software-quality measurements can support fault prediction and quality forecasting [8, 9]. Relevant model families include LSTM and GRU recurrent networks [10], LTSF-Linear models [11], and patch-based architectures such as PatchMLP [12] and PatchTST [13].

DELTX will compare these candidates using repository histories containing the four quality scores alongside AI-authorship, churn, structural, and static-analysis features. The final model will be selected according to MAE and RMSE on validation data, with separate evaluation of its ability to generalize to unseen repositories.

2.4 Explainable Artificial Intelligence

The final research component is explainability. SHAP uses Shapley-value principles to quantify the contribution of input variables to a model prediction [5]. Within DELTX, SHAP is intended to explain the forecasting model rather than source-code semantics directly. A predicted decline can therefore be traced to historical channels such as changes in quality scores, AI confidence, churn, centrality, issue densities, complexity, or duplication.

Chapter 3

Research Objectives / Questions

3.1 Research Objectives

1. **AI Code Detection Methodology.** To operationalize an AI Code Detection methodology using the published DroidDetect-Base-Binary detector to estimate AI-authorship confidence for newly introduced Python code and aggregate detector outputs, weighted by the number of code lines, into a commit-level AI-authorship signal.
2. **Quality Mapping and Weighting Mechanism.** To formulate a mapping and weighting mechanism that translates granular SonarQube rule violations into four primary ISO/IEC 25010 dimensions (Maintainability, Correctness, Security, Efficiency), explicitly factoring in issue severity, occurrence frequency, code churn, and call-graph centrality.
3. **Conflict-Resistant Quality Scoring System.** To engineer a conflict-resistant Quality Scoring System, inspired by hierarchical frameworks such as SQUALE, that computes normalized scores on a 0–100 scale for each overarching quality dimension, with explicit penalization of outliers.
4. **Predictive Modeling and Model Selection.** To compare forecasting models using multivariate historical metric time series and select the best-performing model for forecasting future quality-score trajectories with statistically robust confidence intervals.
5. **Interpretation Layer with SHAP Integration.** To integrate a comprehensive Interpretation Layer utilizing SHAP methodologies to translate numerical quality forecasts into semantic, human-readable insights, benchmarked against empirically derived industry thresholds.

6. Dataset Construction and Empirical Foundation. To design and construct a temporally structured, chronologically mined dataset from mature open-source Python repositories, pairing probabilistic AI-authorship signals with continuous SonarQube quality metrics at discrete commit checkpoints, to serve as the empirical foundation for training and evaluating the predictive modeling engine.

Chapter 4

Research Approach and Methodology

4.1 System Architecture Overview

DELTX is implemented as a multi-stage Python pipeline over repository history and a local Dockerized SonarQube installation. The stages are intentionally decoupled so that authorship detection, extraction, scoring, forecasting, and interpretation can be tested independently.

1. Extract meaningful Python checkpoints and repository-evolution features.
2. Compute a commit-level AI-authorship confidence signal using the existing detector.
3. Scan each Python snapshot with SonarQube and convert issues/measures to four SQUALE-based quality targets.
4. Construct time-series windows and train forecasting models.
5. Explain selected forecasts using SHAP.

4.2 Dataset Construction and Checkpoint Selection

The dataset represents repository evolution as a chronological sequence of checkpoints, with one observation for each commit that introduces a meaningful change to Python source code. Changes are identified by comparing each non-root commit with its **first parent**; the root commit is compared with an empty repository state. Restricting the analysis to Python provides a consistent basis for authorship and software-quality measurements within a single programming language.

Checkpoint selection uses abstract syntax tree (AST) comparison to distinguish changes in program structure from changes that leave the normalized syntax representation unchanged. The following commit types are excluded:

- commits that change only non-Python files such as README files, documentation, images, configuration files, lock files, or other assets;
- Python changes that add or modify only comments;
- Python changes that modify only recognized module, class, function, or asynchronous-function docstrings;
- whitespace-only and formatting-only Python changes;
- pure Python-file renames whose normalized program contents are unchanged; and
- other changes whose normalized Python syntax is semantically unchanged under the AST-based comparison.

Commits are retained when they introduce changes to executable program structure, including changes to a single line, additions or deletions of Python files, and renames accompanied by substantive code changes. When historical Python source cannot be parsed, conservative token-based comparison is used to reduce the risk of excluding relevant changes. This selection strategy focuses the longitudinal dataset on meaningful software evolution events.

The selected repository states are examined as historical snapshots. For each checkpoint, code churn is measured from Python lines added and deleted relative to the first parent. Structural impact is represented by an import-dependency graph, in which nodes correspond to Python files and edges represent imports within the repository. PageRank provides a centrality measure that is included as a dataset feature and used to account for the structural importance of affected files during issue weighting.

4.3 AI Authorship Detection Methodology

The current implementation uses DroidDetect-Base-Binary [6]. For each non-root checkpoint, Deltx compares the commit with its **first parent** and scopes authorship detection to the **new Python code introduced by that commit**. Unchanged pre-existing code is excluded from this authorship measurement, and deleted code has no current content to classify. This makes `ai_confidence_pct` an authorship signal

for the code being introduced at that evolution step rather than a score for the entire repository state.

The rationale for this scope is that integrating newly written code into an existing repository does not change whether that code was human- or AI-authored. AI authorship is therefore measured on the newly introduced code itself. The resulting code-level probabilities are aggregated using a code-line-weighted average to obtain the commit-level `ai_confidence_pct` in the range 0–100, which occupies numeric feature index 4.

A probabilistic AI-authorship signal is produced independently from the quality-scoring engine. The AI signal is not used as a penalty in SQUALE; it serves as an evolutionary driver for the forecasting stage.

4.4 Repository Quality Assessment

At each selected checkpoint, SonarQube analyzes the **complete Python repository snapshot** to assess software quality after the commit. This captures issues in newly introduced code and those arising from its interaction with the existing codebase. The findings are aggregated using SQUALE to obtain repository-level Maintainability, Correctness, Security, and Efficiency scores.

Measurements include NCLOC, cognitive complexity, duplicated-lines density, and maintainability remediation effort. Issue weighting incorporates file centrality, measured using PageRank over the Python import graph, and historical file volatility derived from earlier commits. Missing measures are treated as unavailable rather than as zero. The SonarQube version and quality-profile configuration are recorded for reproducibility.

The dataset combines **AI confidence for newly introduced Python code** with **quality scores for the complete repository state**. Quality changes associated with a commit are assessed by comparing its scores with those of its first parent.

4.5 Quality Mapping

The implemented mapping is:

$$\begin{aligned} \text{MAINTAINABILITY} &\rightarrow \text{score_maintainability}, \\ \text{RELIABILITY} &\rightarrow \text{score_correctness}, \\ \text{SECURITY} &\rightarrow \text{score_security}. \end{aligned}$$

A single Sonar issue can affect more than one quality, and each quality retains its own severity. For Efficiency, SonarQube does not expose a dedicated software quality. Deltx therefore uses active Python rules whose authoritative Clean Code attribute is **EFFICIENT**. This is additive: an issue can influence Reliability/Correctness and Efficiency simultaneously. The Efficiency score is a static-analysis-derived proxy and must not be interpreted as direct runtime performance measurement.

4.6 Dynamic Issue Weighting

For a rule count at checkpoint t , the issue density and bounded frequency signal are

$$\text{density}(r, t) = \frac{\text{count}(r, t)}{\max(\text{NCLOC}_t, 1)} \times 1000, \quad (4.1)$$

$$F'(r, t) = \frac{\ln(1 + \text{density})}{1 + \ln(1 + \text{density})}. \quad (4.2)$$

For a file f , centrality is represented by the within-repository empirical percentile of its PageRank:

$$C'(f, t) = \text{ECDF}(PR(f, t)). \quad (4.3)$$

Historical volatility is

$$V(f, t) = \sum_{k=1}^H \delta^{k-1} \frac{\text{added}(f, t-k) + \text{deleted}(f, t-k)}{\max(\text{previous_loc}(f, t-k), 1)}, \quad (4.4)$$

with bounded transform

$$CH'(f, t) = \frac{\ln(1 + V)}{1 + \ln(1 + V)}. \quad (4.5)$$

The dynamic weight for issue i mapped to quality dimension d is

$$W(i, d, t) = M(i, d)S(i, d) [1 + \rho_d (\alpha_d F'_i + \beta_d C'_i + \gamma_d CH'_i)]. \quad (4.6)$$

The current research baseline uses equal α , β , and γ values of $1/3$, with $\rho = 1$. These are provisional baseline values and not validated Python-optimal parameters.

4.7 Individual Marks and SQUALE Aggregation

The issue weight is converted to a common 0–3 mark:

$$IM(i, d) = 3 \left[1 - \text{clip} \left(\frac{W}{5(1 + \rho_d)}, 0, 1 \right)^{\kappa_d} \right]. \quad (4.7)$$

For individual marks IM_j and positive aggregation weights ω_j , Deltx uses

$$M_d = -\frac{\ln\left(\frac{\sum_j \omega_j \lambda_d^{-IM_j}}{\sum_j \omega_j}\right)}{\ln(\lambda_d)}, \quad (4.8)$$

followed by

$$\text{score}_d = \text{clip}\left(\frac{100}{3}M_d, 0, 100\right). \quad (4.9)$$

The exponential mean gives lower marks greater influence than arithmetic averaging, reducing the extent to which higher marks can compensate for poorer quality.

4.8 Maintainability Composite

Maintainability is not based only on issue impacts. First, all MAINTAINABILITY issue marks are aggregated into a 0–3 issue-practice mark. Three additional project-level practices are transformed using

$$\text{metric_mark}(x) = \frac{3}{1 + (x/\tau)^k}. \quad (4.10)$$

The four outer practices are:

1. the aggregated MAINTAINABILITY issue-practice mark;
2. technical debt per KLOC, $\text{sqale_index}/\max(\text{NCLOC}, 1) \times 1000$;
3. cognitive complexity per KLOC;
4. duplicated-lines density, used directly because it is already normalized.

These four practice marks are then combined with the nonlinear SQUALE aggregation to obtain `score_maintainability`. The current baseline uses $\lambda = 9$, debt threshold $\tau = 1000$ minutes/KLOC, complexity threshold $\tau = 100$ per KLOC, and duplication threshold $\tau = 5\%$, with unit shape/aggregation weights. These are research baselines rather than empirically validated thresholds.

4.9 Dataset Vector

Each generated CSV row begins with the two identifiers `repository` and `commit_hash`, followed by the 15 numeric features in Table 4.1. The identifiers are used for grouping and traceability and are excluded from model inputs.

Table 4.1: Deltx commit-level data vector.

Index	Classification	Variable	Description
–	Identifier	<code>repository</code>	Repository directory name used for grouping and traceability.
–	Identifier	<code>commit_hash</code>	Full Git commit object identifier.
0	Predictive Target	<code>score_maintainability</code>	SQUALE score using MAINTAINABILITY impacts plus debt/KLOC, cognitive complexity/KLOC, and duplication density.
1	Predictive Target	<code>score_correctness</code>	SQUALE score from Sonar RELIABILITY impacts.
2	Predictive Target	<code>score_security</code>	SQUALE score from Sonar SECURITY impacts.
3	Predictive Target	<code>score_efficiency</code>	SQUALE score from active Python rules with Clean Code attribute EFFICIENT.
4	Evolutionary Driver	<code>ai_confidence_pct</code>	LOC-weighted commit-level AI authorship confidence in [0, 100].
5	Evolutionary Driver	<code>loc_added</code>	Python lines added in the checkpoint.
6	Evolutionary Driver	<code>loc_deleted</code>	Python lines deleted in the checkpoint.
7	Evolutionary Driver	<code>files_modified_count</code>	Number of modified Python files.
8	Topological Impact	<code>avg_pagerank centrality</code>	Mean raw PageRank of meaningfully changed Python files.
9	Static Analysis Signal	<code>density_blocker_issues</code>	Blocker issue density per KLOC.
10	Static Analysis Signal	<code>density_critical_issues</code>	Critical/high issue density per KLOC under the fixed dataset bucket mapping.
11	Static Analysis Signal	<code>density_major_issues</code>	Major/medium issue density per KLOC.
12	Static Analysis Signal	<code>density_minor_issues</code>	Minor/low issue density per KLOC.
13	Static Analysis Signal	<code>cognitive_complexity</code>	Sonar project cognitive complexity.

Index	Classification	Variable	Description
14	Static Analysis Signal	<code>duplication_density</code>	Sonar duplicated-lines density percentage.

4.10 Predictive Modeling and Interpretation

Forecasting uses historical windows of the 15 numeric channels and predicts future values of the four target scores. Repository identifiers are used to prevent windows from crossing repository boundaries. Time-aware splitting is required so future checkpoints are not exposed to training through overlapping windows. The final selected forecasting model will be interpreted using SHAP to identify the historical channels and time windows that contribute most strongly to each forecast.

Chapter 5

Experiments and Preliminary Results

5.1 Repositories Analysed

At the current interim stage, datasets have been extracted from the 15 Python repositories listed in Table 5.1. The table also summarizes the role of each repository and the size of its public commit history.

Table 5.1: Repositories currently analysed with the Deltx data pipeline.

No.	Repository	Repository type / purpose	Commits	Source
1	Pyevolve	Evolutionary-computation library/framework for genetic algorithms and genetic programming	168	perone/Pyevolve
2	PHC	Research implementation for physics-based humanoid control and simulated avatars (ICCV 2023)	200	ZhengyiLuo/PHC
3	Sloppy-xml-py	Parsing library for malformed XML	37	mitsuhiko/sloppy-xml-py
4	Markupsafe	Web/security utility library for safe HTML/XML markup escaping	869	pallets/markupsafe
5	Itsdangerous	Security and serialization library for signing data passed through untrusted environments	677	pallets/itsdangerous
6	Typer-cli	Developer CLI utility for running Typer scripts with shell completion	126	tiangolo/typer-cli

No.	Repository	Repository type / purpose	Commits	Source
7	FastAPI-cli	Developer/web CLI tool for serving and managing FastAPI applications	684	fastapi/fastapi-cli
8	Flask	Production web framework / Python microframework for web applications	5,557	pallets/flask
9	Requests	Networking library / HTTP client	6,495	psf/requests
10	qpbenchmark	Scientific benchmarking toolkit for quadratic-programming solvers	1,034	qpsolvers/qpbenchmark
11	dkist	Scientific data library for accessing and processing DKIST solar-observatory data	1,003	DKISTDC/dkist
12	Rompy	Scientific modelling library for coastal-ocean and wave-model setup, execution, and analysis	1,000	rom-py/rompy
13	pyhf-dvp	Scientific/statistical research library for HistFactory-style fitting and inference	1,096	CIERA-Transients/pyhf-dvp
14	Python-scrapinghub	API client library for the Scrapinghub platform	800	scrapinghub/python-scrapinghub
15	Td-client-python	Data-platform API client library for Treasure Data	800	treasure-data/td-client-python

Commit counts are the default-branch history counts displayed by GitHub on 6 October 2026.

5.1.1 Dataset Extraction Runtime and Compute Cost

Dataset construction is computationally expensive because each retained checkpoint requires repository-history processing, AI-authorship analysis, a complete Python snapshot scan through the local Dockerized SonarQube stack, issue/metric collection, and SQUALE aggregation. In the current experiments, a repository with approximately 1,000 commits required about **7 hours** of wall-clock processing time under the available hardware and filtering configuration. The substantially larger Flask repository required approximately **3 days** (about 72 wall-clock hours) to complete its dataset extraction.

These figures should be treated as observed compute-cost estimates rather than fixed complexity bounds. Runtime varies with the number of commits retained after semantic filtering, repository size at each checkpoint, the number and size of Python files, AI-detection workload, SonarQube analysis time, Docker and storage

performance, and the hardware used. Dataset-generation runs have been executed both on the **local PCs** and on an **Azure virtual machine** funded using free Azure for Students credits. Consequently, the principal cost incurred during the interim experiments has been compute time rather than direct paid cloud expenditure.

5.2 Repository-Level Exploratory Analysis

The extracted datasets were explored using three complementary plots: a commit-sequence line chart to inspect temporal variation, a correlation heatmap to inspect linear association between the plotted signals, and pairwise scatter plots to inspect distributions and relationships that may not be obvious from the line chart alone. The complete supplied visualization set is included in Appendix A.

The plots show that repository histories differ substantially in length and in the stability of their quality trajectories. AI-confidence is often highly variable across commits, while the quality scores generally evolve on a different temporal scale. This supports modeling quality as a multivariate temporal problem rather than assuming a direct contemporaneous relationship between AI confidence and a quality score. The exploratory plots are descriptive only; they are not used as evidence of causation.

Complete line-chart, heatmap, and scatter-plot sets are currently available for 12 of the 15 analysed repositories. Visualizations are not currently available for Rompy, pyhf-dvp, and Python-scrapinghub; no plots are fabricated for those repositories.

5.3 Forecasting Experiments

The first completed forecasting baseline is a per-repository ARIMA experiment for `score_maintainability`. It provides a classical baseline for the broader comparison of forecasting models, which will use the multivariate feature vector and evaluate all four quality targets. The ARIMA results provide preliminary evidence about short-term predictability of the Maintainability trajectory.

5.3.1 Rolling-Origin Evaluation Protocol

The ARIMA experiments use **time-series cross-validation with a rolling origin** (walk-forward validation). The commit sequence is kept in chronological order and is first divided into an initial historical training region and a later test region (approximately an 80/20 temporal split in the current runs). At each test step, the model is fitted or updated using all observations available up to commit c_t , forecasts exactly one step ahead to c_{t+1} , then receives the true c_{t+1} value before forecasting the next

commit. This repeats until the test region is exhausted. No future observation is made available before its prediction.

The protocol is designed to mimic practical Deltx usage: developers would request the expected quality of the next commit state, observe the actual repository state when that commit arrives, and then update the forecasting history. Performance is measured using MAE and RMSE and compared with a **naive persistence/random-walk baseline**, which predicts that the next Maintainability score will equal the latest observed score. The approach is appropriate for statistical per-repository models such as ARIMA because it adapts after structural changes in the history. Its main costs are repeated model updates and the possibility that an ever-expanding history gives excessive influence to obsolete repository behaviour; a sliding window can be investigated for highly non-stationary projects.

For models trained across multiple repositories, generalization will be assessed by holding out entire repositories and evaluating forecasting windows on projects not used for training. Multi-step strategic forecasting, where the system must predict many commits ahead without receiving intervening observations, will also require a separate fixed-horizon evaluation.

5.3.2 Preliminary ARIMA Results for Maintainability

Table 5.2 reports the available rolling-origin results for `score_maintainability`. The corresponding forecast-vs-actual plots are included in Appendix B. Lower MAE and RMSE indicate better forecasts.

Table 5.2: ARIMA one-step-ahead Maintainability forecasting results compared with the naive persistence baseline.

Repository	ARIMA MAE	ARIMA RMSE	Naive MAE	Naive RMSE	Result
DKIST	0.15	0.22	2.42	2.62	Better
FastAPI-cli	0.48	0.70	0.44	0.53	Worse
Flask	0.06	0.17	0.57	0.66	Better
Itsdangerous	0.22	0.61	2.23	2.67	Better
Markupsafe	0.91	2.05	3.00	3.53	Better
PHC	0.06	0.18	0.08	0.24	Better
Pyevolve	0.14	0.20	0.23	0.24	Better
qpbenchmark	0.19	0.31	1.75	2.22	Better
Requests	0.78	3.44	20.23	23.96	Better
Sloppy-xml-py	0.25	0.31	0.15	0.20	Worse
Td-client-python	0.38	0.61	2.76	3.13	Better
Typer-cli	1.62	1.63	0.08	0.11	Worse

ARIMA outperformed the naive baseline on both MAE and RMSE for **9 of the 12 repositories** with available forecasting outputs. The strongest improvements are visible for DKIST, Flask, Itsdangerous, qpbenchmark, Requests, and Td-client-python, where the ARIMA forecast follows changes in the held-out Maintainability trajectory much more closely than a fixed persistence forecast. Markupsafe also shows a clear improvement, while PHC and Pyevolve show smaller but consistent gains.

The three exceptions are FastAPI-cli, Sloppy-xml-py, and Typer-cli. FastAPI-cli is close to the persistence baseline but is slightly worse on both metrics. Sloppy-xml-py has a very short history in the available experiment, and the naive forecast is more accurate than the fitted ARIMA trajectory. Typer-cli is the clearest failure case: the held-out series is nearly flat, so persistence is extremely strong (MAE 0.08, RMSE 0.11), whereas ARIMA introduces a systematic offset (MAE 1.62, RMSE 1.63). These cases are important because they show that an autoregressive model should not be assumed to outperform a simple baseline on every repository.

Across the 12 available outputs, the median ARIMA error is 0.235 MAE and 0.46 RMSE, compared with median naive errors of 1.16 MAE and 1.44 RMSE. These aggregate values are descriptive only; repository-level results remain the primary evidence because commit histories differ substantially in length, volatility, and structural breaks.

5.4 Preliminary Findings

At this stage, the empirical results support four limited conclusions. First, the implemented pipeline can generate longitudinal Python quality datasets over repositories with very different history lengths. Second, the four quality scores are not reducible to a single common trajectory: their variation differs between repositories and across commits. Third, the AI-authorship signal is sufficiently variable to be useful as a time-series driver, but the exploratory plots alone do not establish that higher AI confidence causes lower software quality. Fourth, the initial ARIMA Maintainability experiment shows that short-term score trajectories are forecastable beyond persistence in many, but not all, repositories: ARIMA beats the naive baseline on 9 of 12 available repository outputs. The failure cases reinforce the need for repository-aware baselines and comparative evaluation of forecasting models rather than assuming that a more complex temporal model always performs better.

Chapter 6

Finalized Evaluation Plan

The evaluation plan covers AI detection, predictive-model accuracy, and explainability, together with direct validation of the implemented scoring and dataset-construction stages.

6.1 AI Code Detection Evaluation

The current implementation uses a published detector rather than training a new detector. File-level and commit-level AI percentage can be reported.

6.2 Dataset Integrity and Reproducibility

A stratified sample of generated rows will be independently audited. Repository/commit identifiers will be used to reproduce the corresponding Git snapshot and verify Python LOC additions/deletions, modified-file counts, Sonar measures, issue counts, and score recomputation. The semantic commit filter will also be manually evaluated on samples of retained and rejected commits to measure whether documentation-only, comment-only, docstring-only, formatting-only, and genuine semantic changes are handled correctly.

For reproducibility, each experimental dataset must retain the SonarQube version, Python quality profile, scoring configuration, detector version, and commit identity. Datasets generated under different analysis versions or quality profiles should be combined only when their measurement configurations are shown to be equivalent.

6.3 Scoring Validation

The four target scores will be evaluated using complementary methods rather than treated as an unquestionable ground truth.

1. **Mathematical invariant tests:** verify boundedness, determinism, monotonic response to severity/frequency/centrality/churn, and the nonlinear SQUALE conflict-resistance property.
2. **Controlled mutation experiments:** introduce one known maintainability, correctness, security, or efficiency degradation at a time and verify the expected directional change while other factors are controlled.
3. **Correctness validation:** use real buggy/fixed Python pairs such as BugsInPy and test whether fixes tend to improve `score_correctness`.
4. **Security validation:** compare Deltx/Sonar findings with known-vulnerability suites such as OWASP Benchmark Python and with independent analyzers such as CodeQL and Bandit.
5. **Maintainability validation:** compare Deltx maintainability rankings with independent measures such as Radon Maintainability Index/complexity and, where applicable, CodeQL quality findings.
6. **Efficiency validation:** for selected performance-related changes, compare the static EFFICIENT-rule proxy with runtime evidence from controlled benchmarks/profilers such as pyperf or Scalene. This tests directional agreement without claiming that the static score is direct runtime measurement.

6.4 Predictive Modeling Accuracy

MAE and RMSE are the primary forecasting metrics:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad (6.1)$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}. \quad (6.2)$$

For statistical per-repository baselines such as ARIMA, SARIMA, or VAR, evaluation will use rolling-origin one-step-ahead validation as described in Section 5.3. This preserves temporal order, continuously incorporates newly observed commits, and compares each model against a naive persistence baseline under the same test region.

If an expanding history becomes harmful on strongly non-stationary repositories, a sliding-window variant will be evaluated.

For forecasting models trained across repositories, the primary generalization experiment will use **whole-repository hold-out**: repositories used for evaluation will not contribute training windows. This tests whether a model transfers to an unseen codebase rather than only memorizing the temporal behaviour of repositories present during training. Performance will be reported separately for Maintainability, Correctness, Security, and Efficiency, with any cross-target average used only as a secondary summary. Long-horizon forecasting, if included as a product objective, will be evaluated separately without rolling access to intervening ground-truth commits.

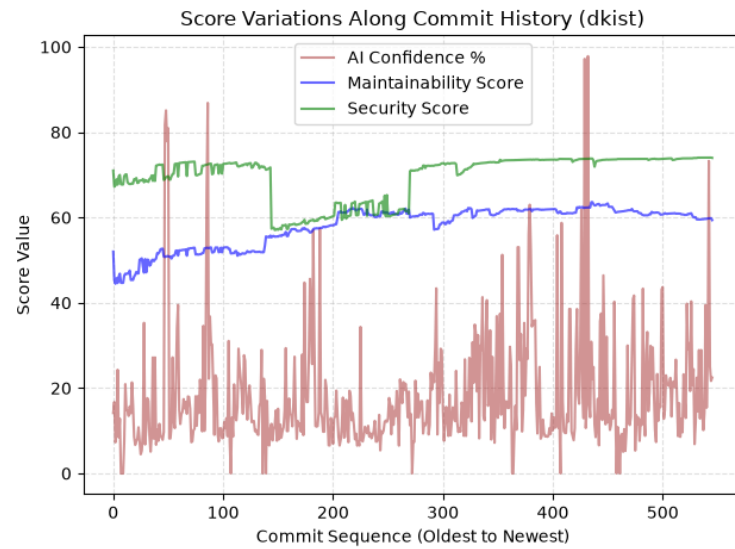
6.5 Explainability and Practical Usefulness

Human expert validation will assess the practical usefulness of SHAP explanations. After selecting the forecasting model, SHAP will be used to identify influential historical channels and time windows. Experienced software engineers will assess whether the explanations are consistent with actual repository changes and whether the reported drivers are actionable. The evaluation should record both the relevance of the explanation and whether it helps a developer identify the source of a predicted quality decline.

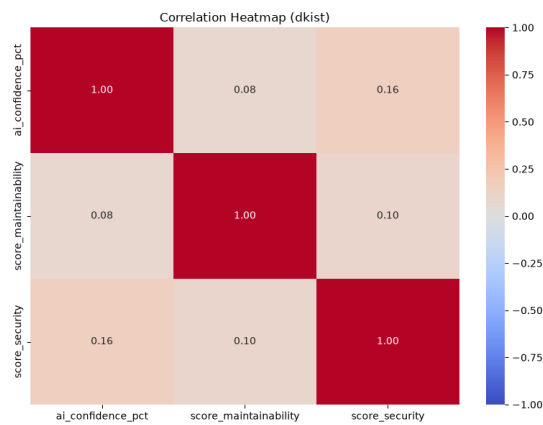
Appendix A

Repository-Level Exploratory Visualizations

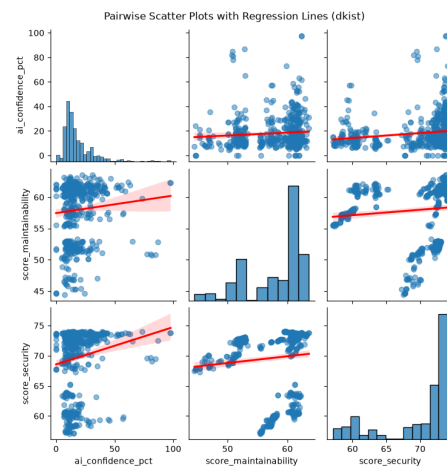
This appendix presents the available repository-level exploratory visualizations generated from the extracted Deltx datasets. Each repository is shown using a commit-history line chart, correlation heatmap, and pairwise scatter plot.



(a) Commit-sequence score variation.

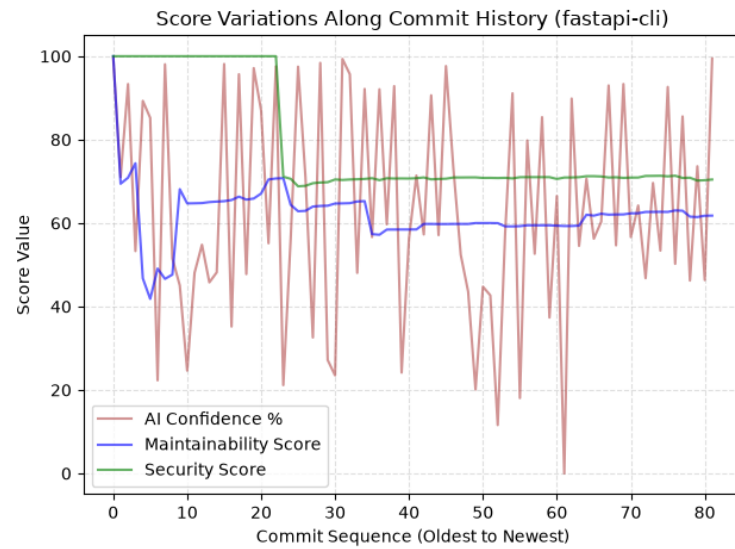


(b) Correlation heatmap.

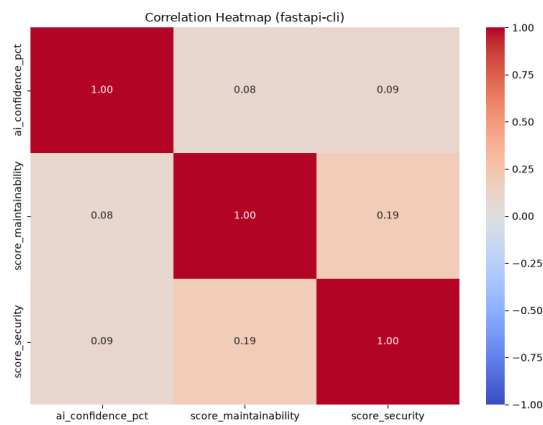


(c) Pairwise scatter visualization.

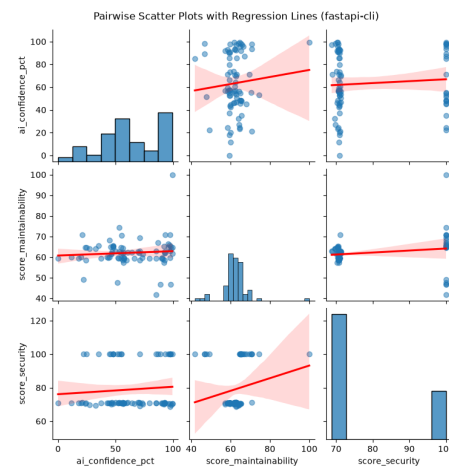
Figure A.1: Exploratory Deltx dataset visualizations for dkist. These figures are the supplied visualization outputs generated from the extracted repository dataset.



(a) Commit-sequence score variation.

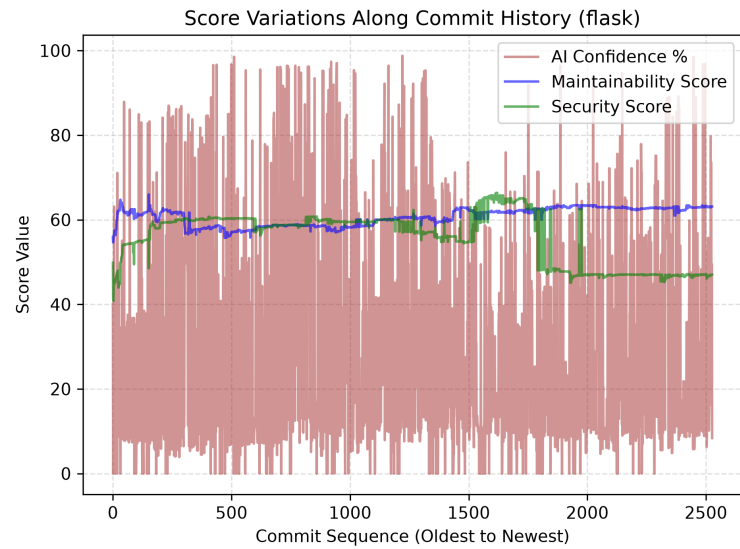


(b) Correlation heatmap.

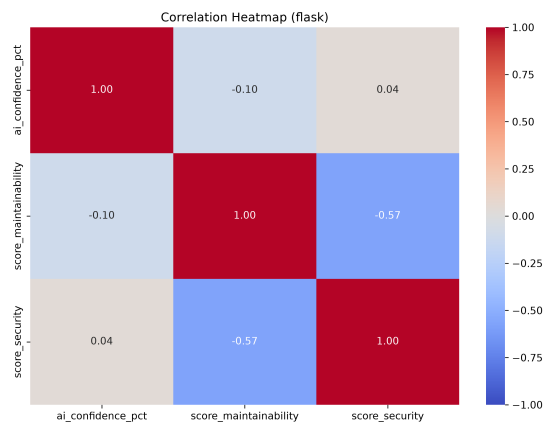


(c) Pairwise scatter visualization.

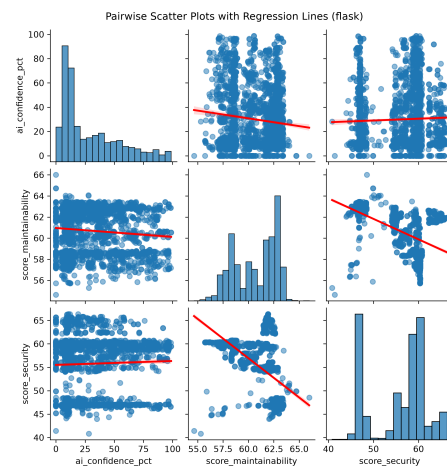
Figure A.2: Exploratory Deltx dataset visualizations for FastAPI-cli. These figures are the supplied visualization outputs generated from the extracted repository dataset.



(a) Commit-sequence score variation.

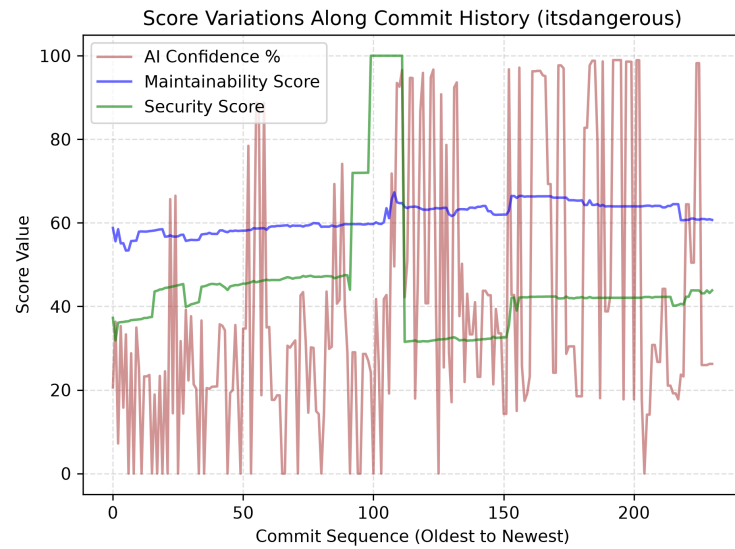


(b) Correlation heatmap.

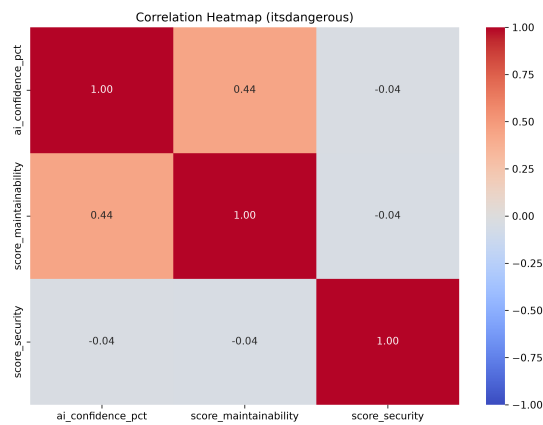


(c) Pairwise scatter visualization.

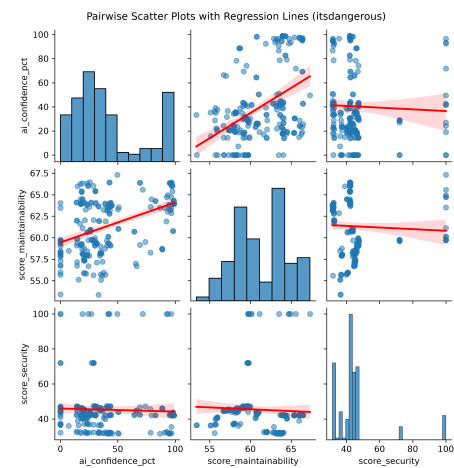
Figure A.3: Exploratory Deltx dataset visualizations for Flask. These figures are the supplied visualization outputs generated from the extracted repository dataset.



(a) Commit-sequence score variation.

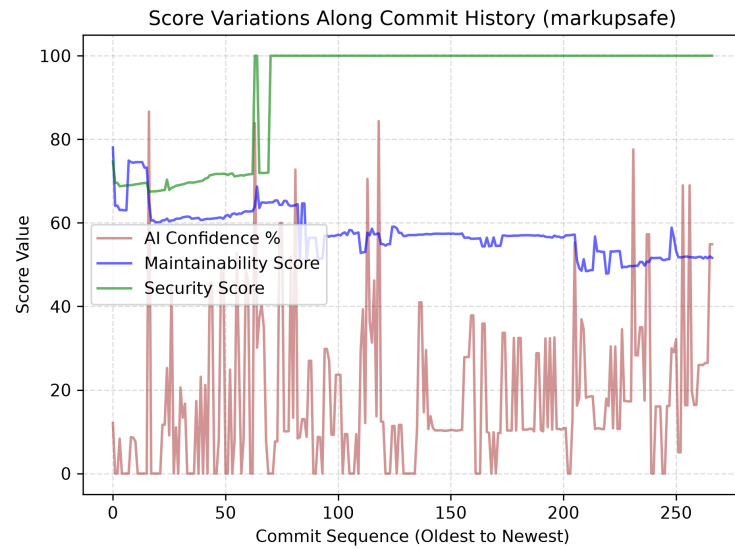


(b) Correlation heatmap.

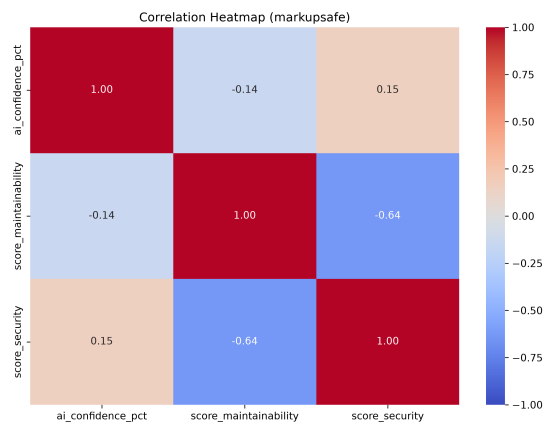


(c) Pairwise scatter visualization.

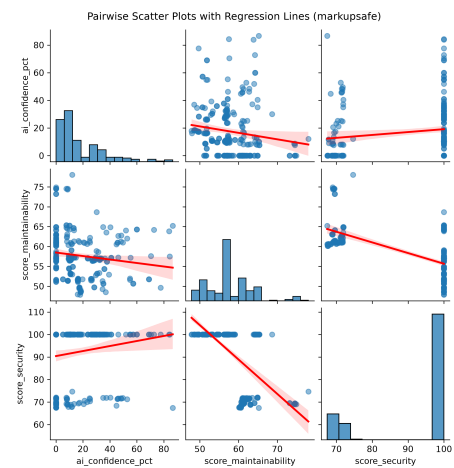
Figure A.4: Exploratory Deltx dataset visualizations for Itsdangerous. These figures are the supplied visualization outputs generated from the extracted repository dataset.



(a) Commit-sequence score variation.



(b) Correlation heatmap.

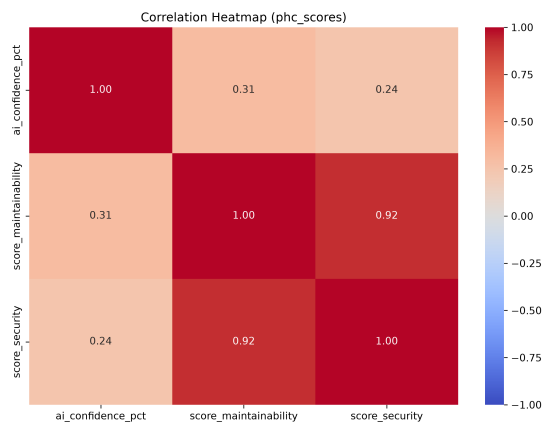


(c) Pairwise scatter visualization.

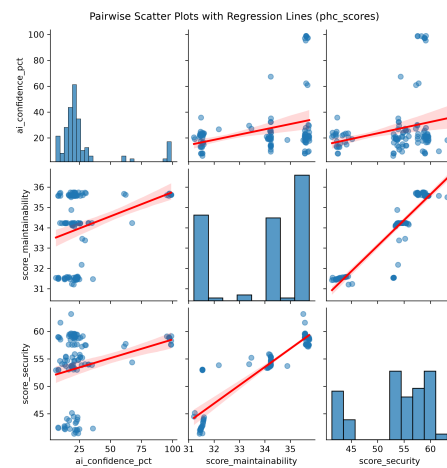
Figure A.5: Exploratory Deltx dataset visualizations for Markupsafe. These figures are the supplied visualization outputs generated from the extracted repository dataset.



(a) Commit-sequence score variation.

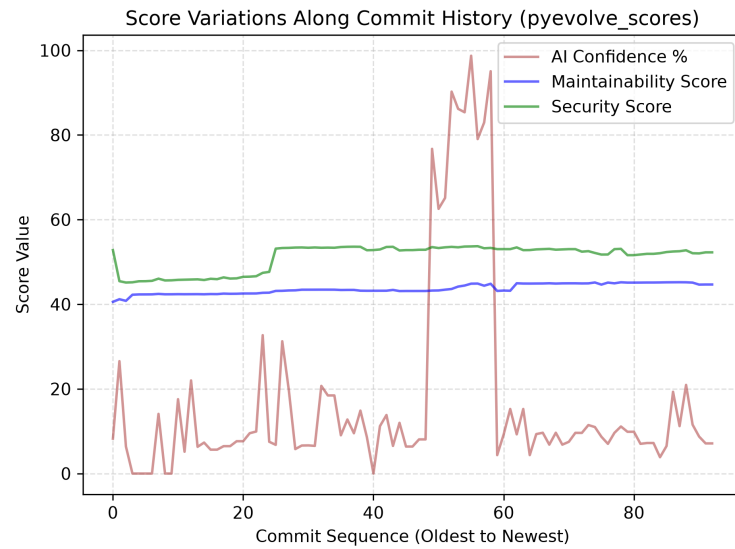


(b) Correlation heatmap.

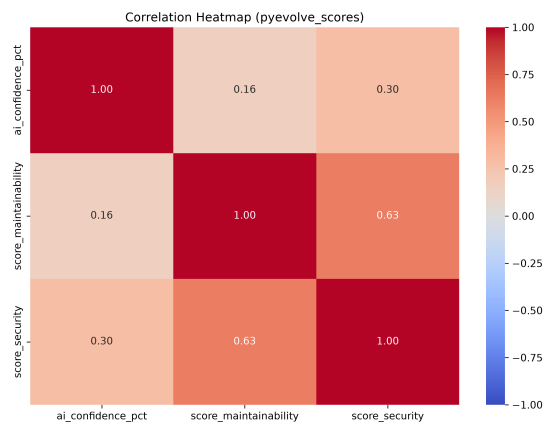


(c) Pairwise scatter visualization.

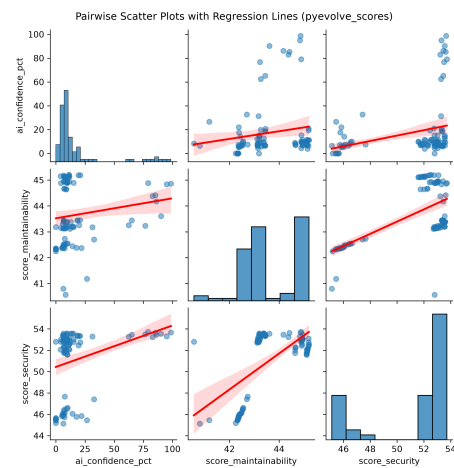
Figure A.6: Exploratory Deltx dataset visualizations for PHC. These figures are the supplied visualization outputs generated from the extracted repository dataset.



(a) Commit-sequence score variation.

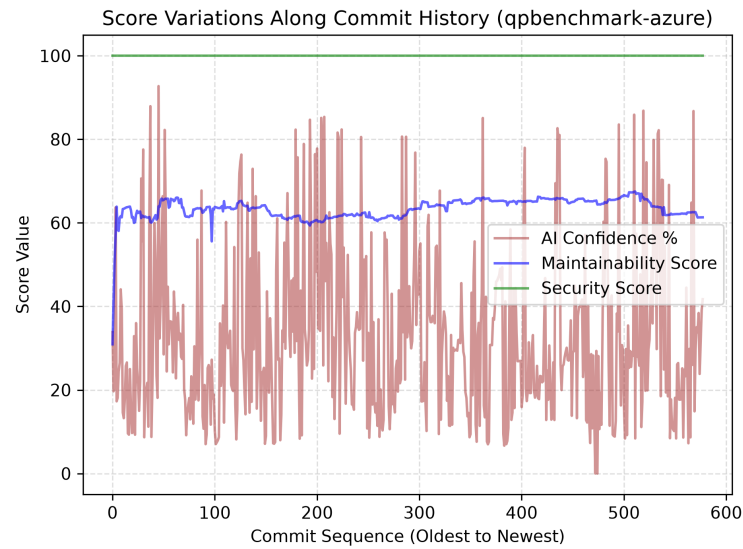


(b) Correlation heatmap.

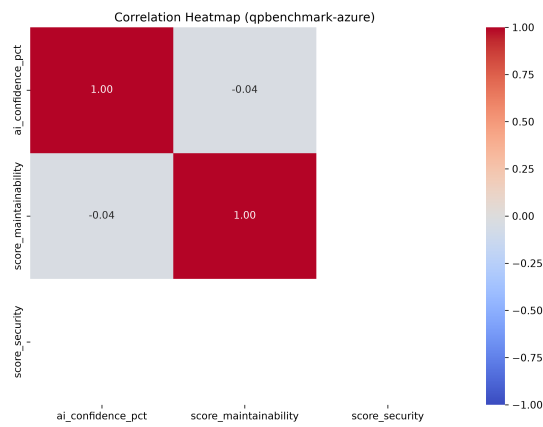


(c) Pairwise scatter visualization.

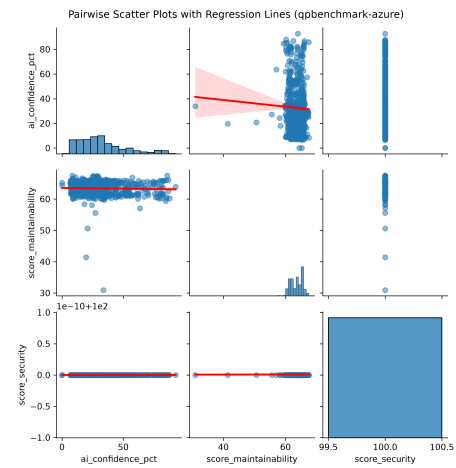
Figure A.7: Exploratory Deltx dataset visualizations for Pyevolve. These figures are the supplied visualization outputs generated from the extracted repository dataset.



(a) Commit-sequence score variation.

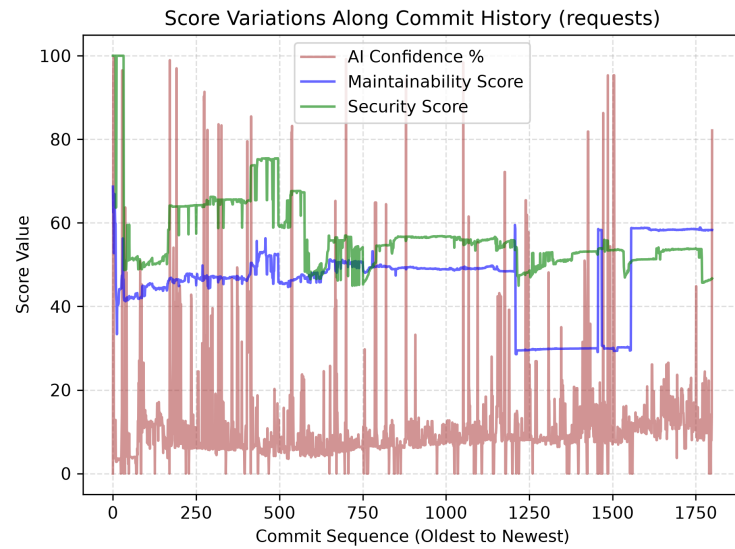


(b) Correlation heatmap.

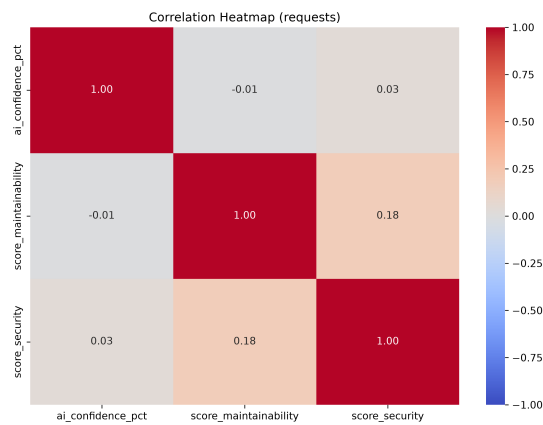


(c) Pairwise scatter visualization.

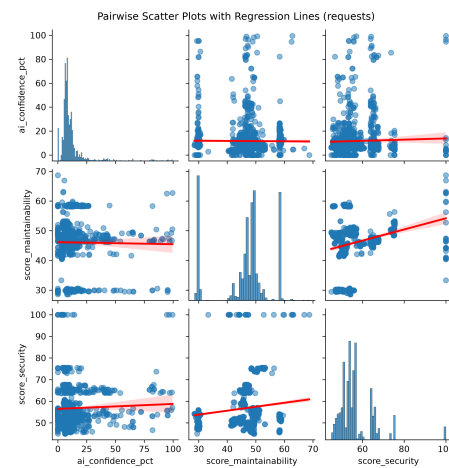
Figure A.8: Exploratory Deltx dataset visualizations for qpbenchmark. These figures are the supplied visualization outputs generated from the extracted repository dataset.



(a) Commit-sequence score variation.

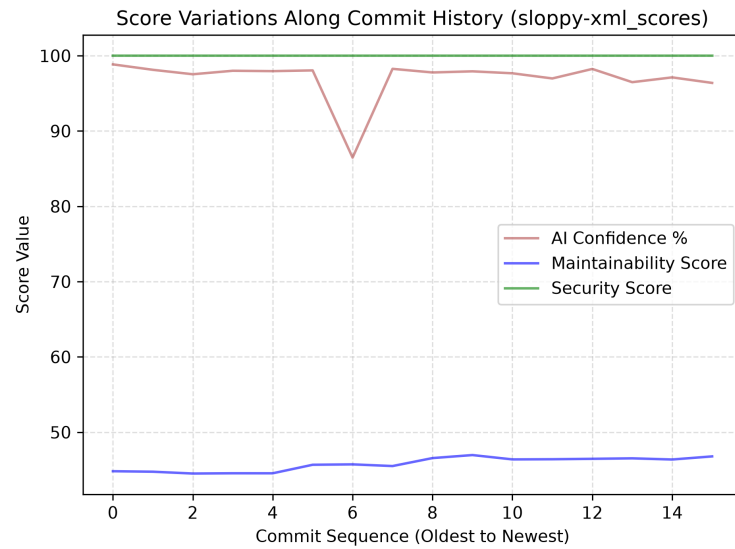


(b) Correlation heatmap.

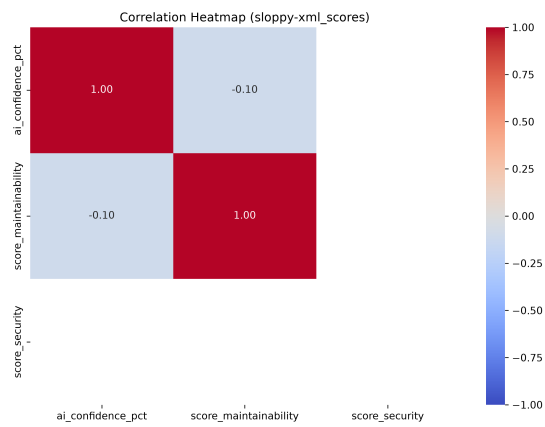


(c) Pairwise scatter visualization.

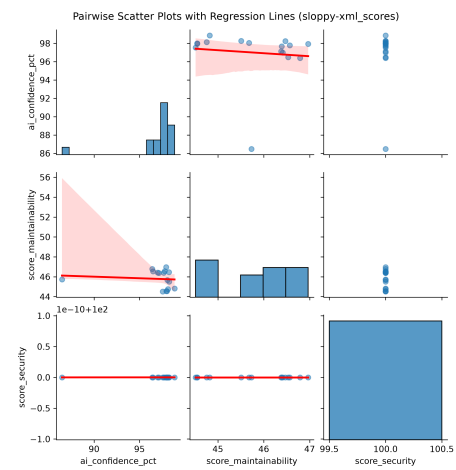
Figure A.9: Exploratory Deltx dataset visualizations for Requests. These figures are the supplied visualization outputs generated from the extracted repository dataset.



(a) Commit-sequence score variation.

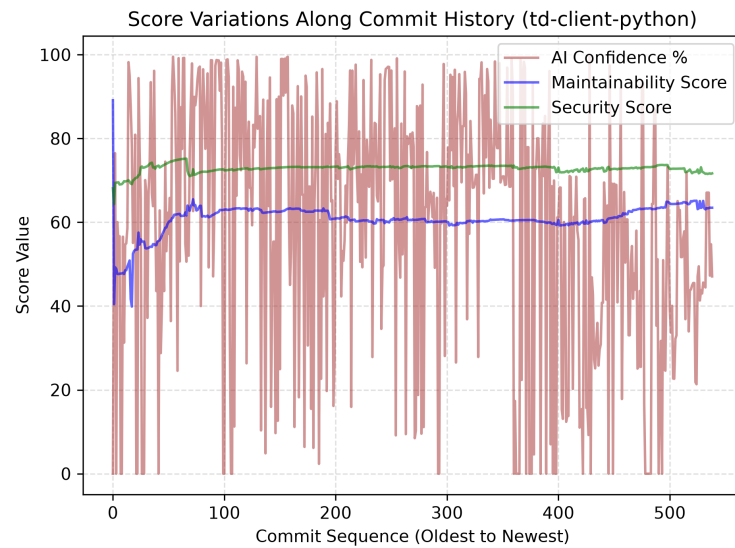


(b) Correlation heatmap.

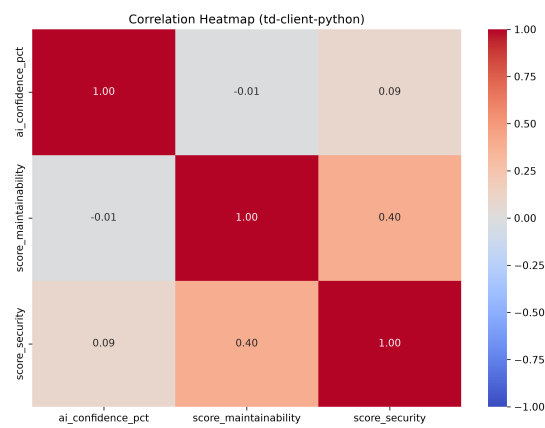


(c) Pairwise scatter visualization.

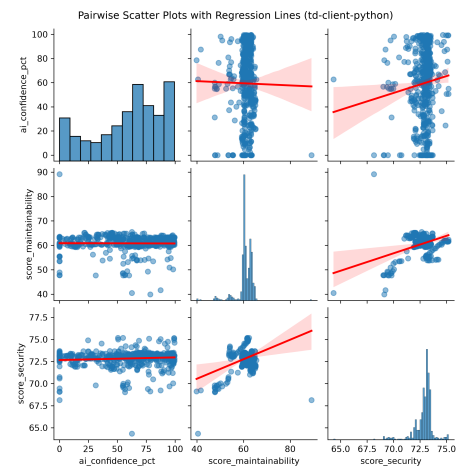
Figure A.10: Exploratory Deltx dataset visualizations for Sloppy-xml-py. These figures are the supplied visualization outputs generated from the extracted repository dataset.



(a) Commit-sequence score variation.

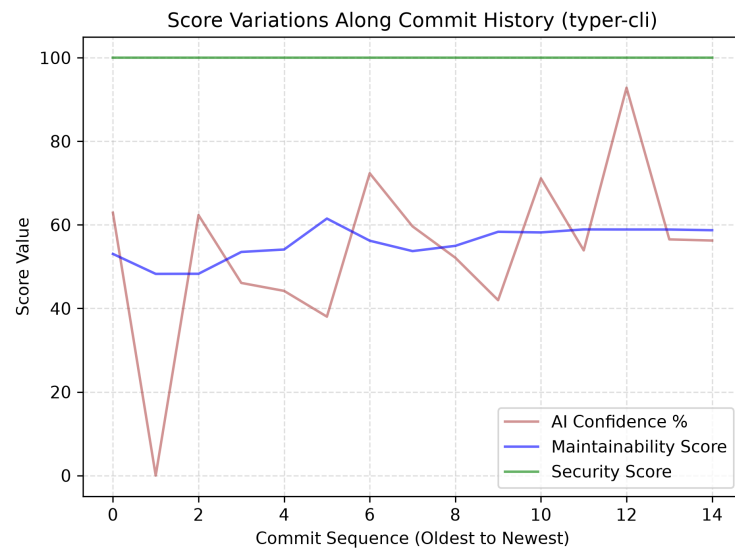


(b) Correlation heatmap.

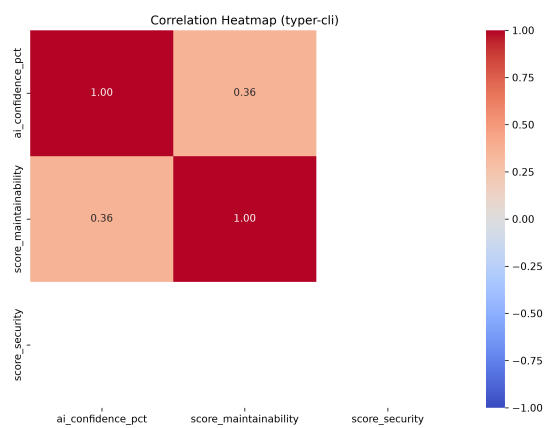


(c) Pairwise scatter visualization.

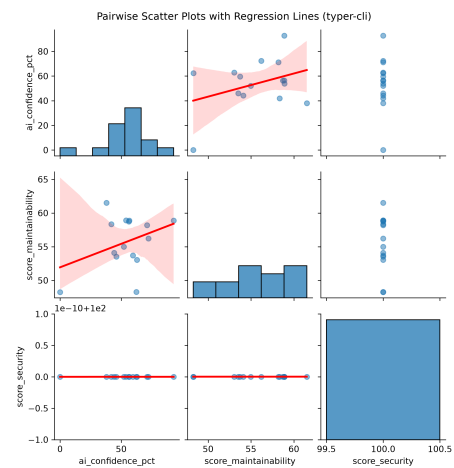
Figure A.11: Exploratory Deltx dataset visualizations for Td-client-python. These figures are the supplied visualization outputs generated from the extracted repository dataset.



(a) Commit-sequence score variation.



(b) Correlation heatmap.



(c) Pairwise scatter visualization.

Figure A.12: Exploratory Deltx dataset visualizations for Typer-cli. These figures are the supplied visualization outputs generated from the extracted repository dataset.

Appendix B

ARIMA Maintainability Forecast Outputs

This appendix presents the available one-step-ahead ARIMA forecast outputs for `score_maintainability`. In each plot, the historical training series is followed by the held-out test region, with ARIMA predictions compared against the actual Maintainability score and the naive persistence forecast.

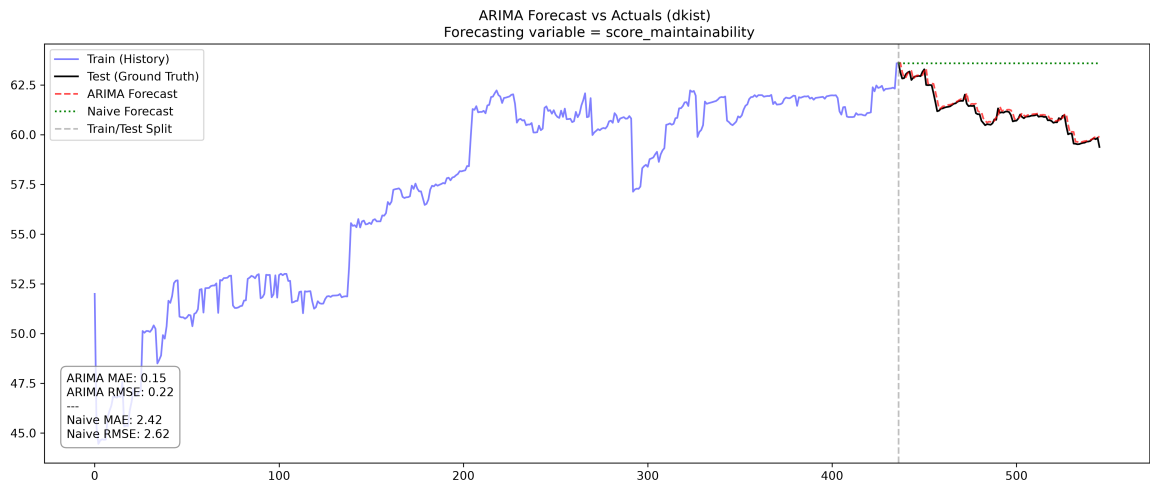
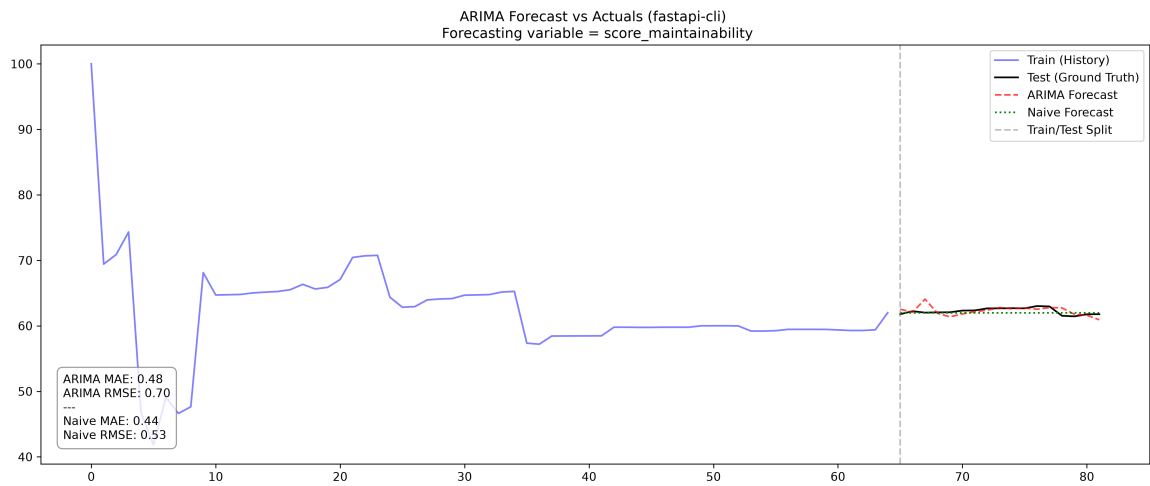


Figure B.1: DKIST ARIMA Maintainability forecast.



FastAPI-cli ARIMA Maintainability forecast.

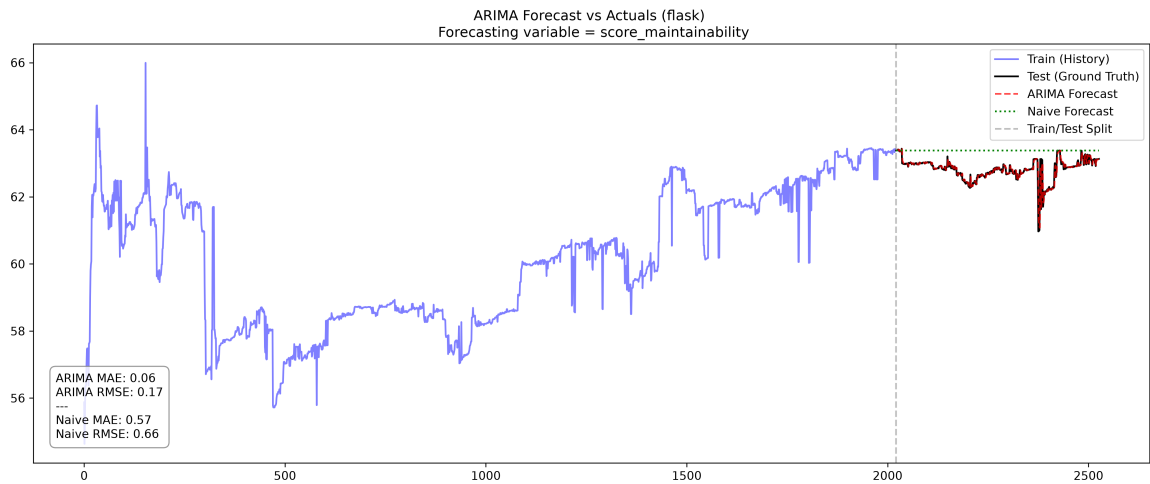
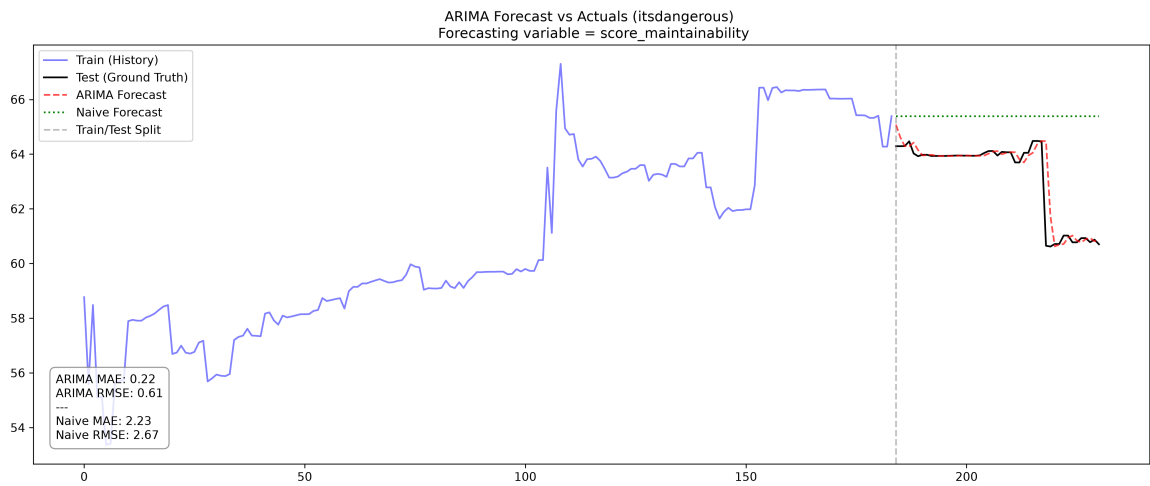


Figure B.2: Flask ARIMA Maintainability forecast.



Itsdangerous ARIMA Maintainability forecast.

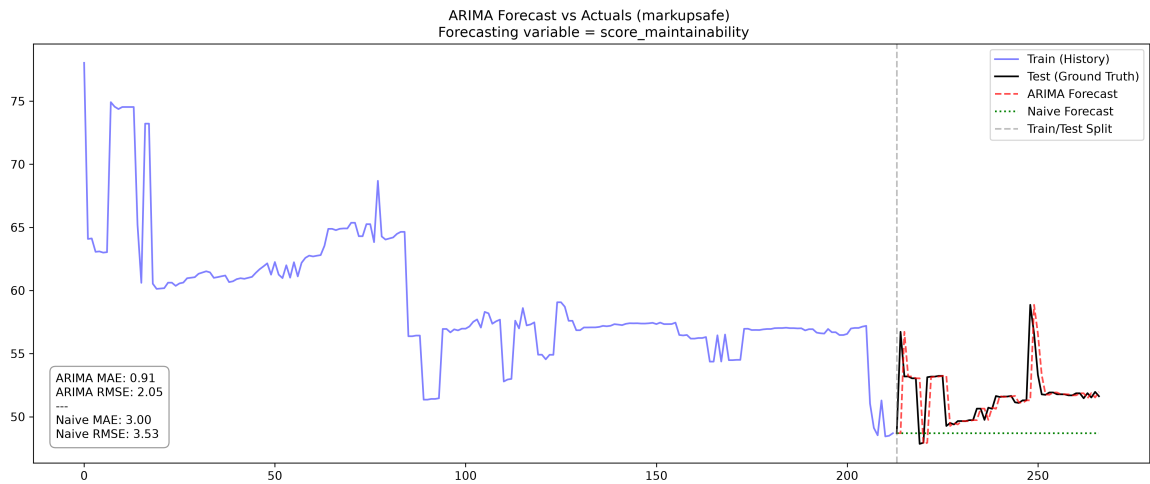
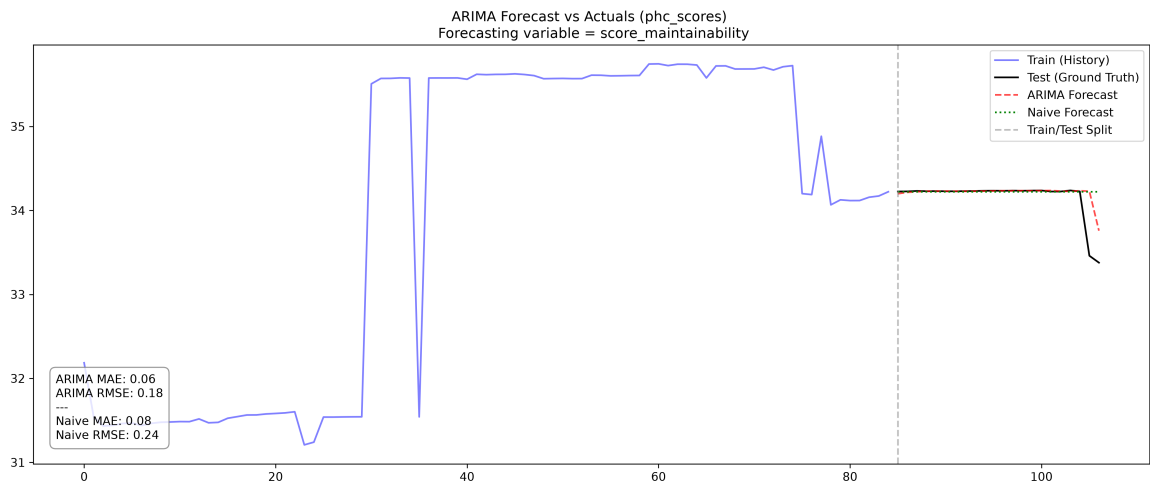


Figure B.3: Markupsafe ARIMA Maintainability forecast.



PHC ARIMA Maintainability forecast.

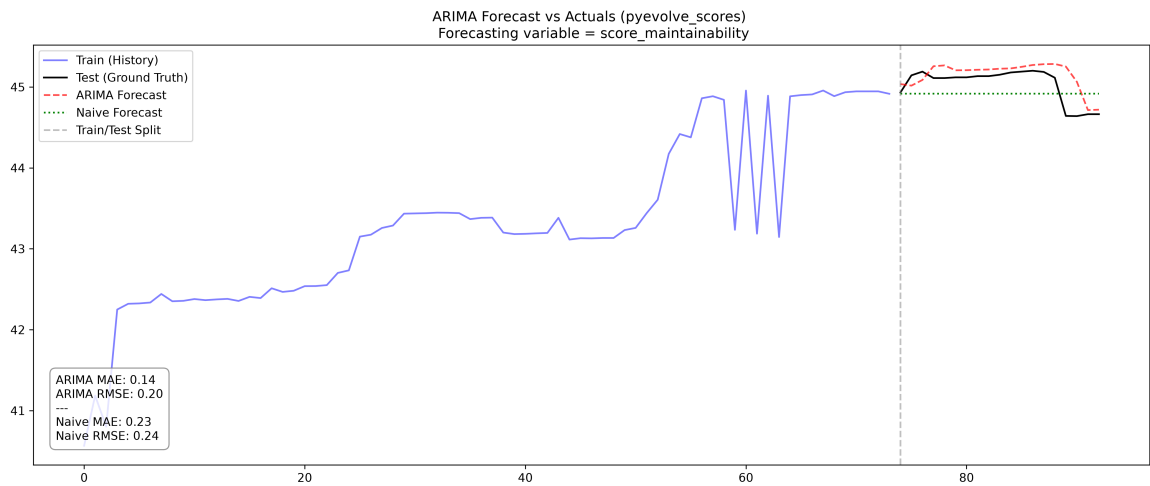
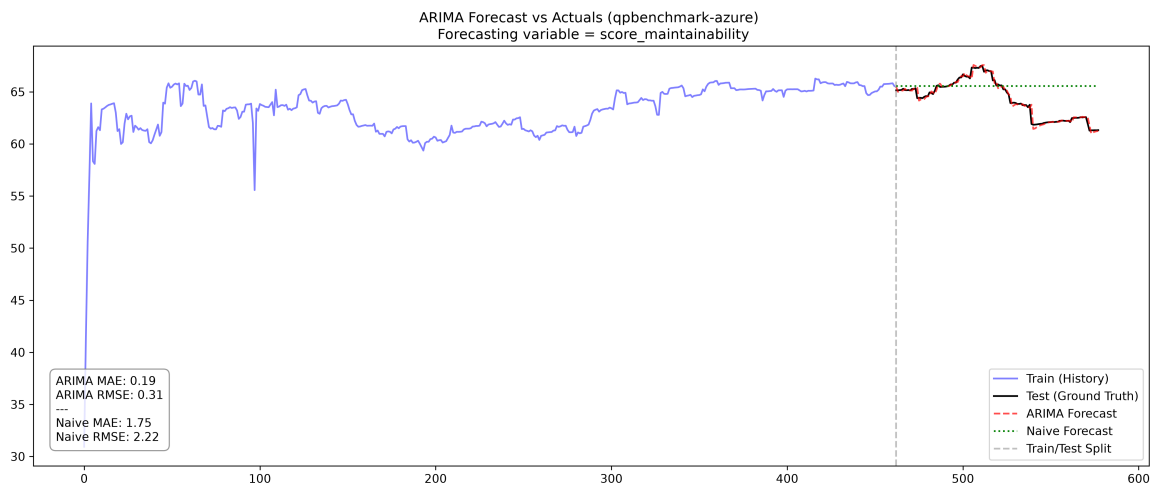


Figure B.4: Pyevolve ARIMA Maintainability forecast.



qpbenchmark ARIMA Maintainability forecast.

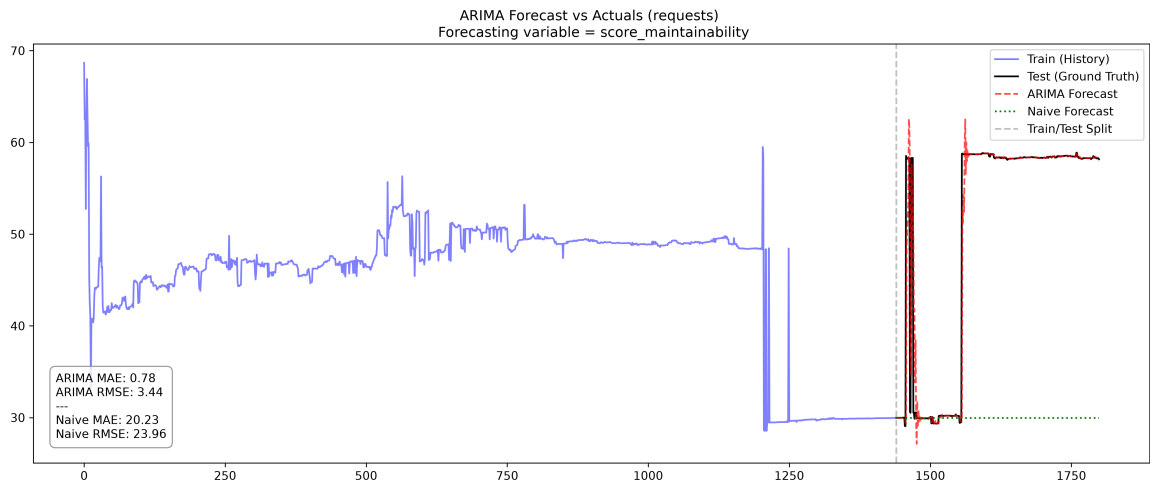
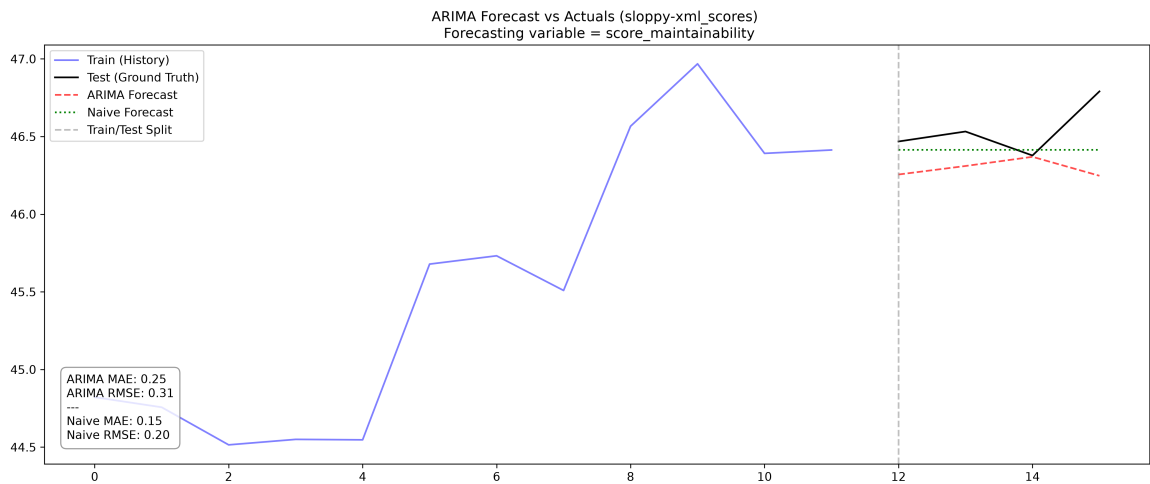


Figure B.5: Requests ARIMA Maintainability forecast.



Sloppy-xml-py ARIMA Maintainability forecast.

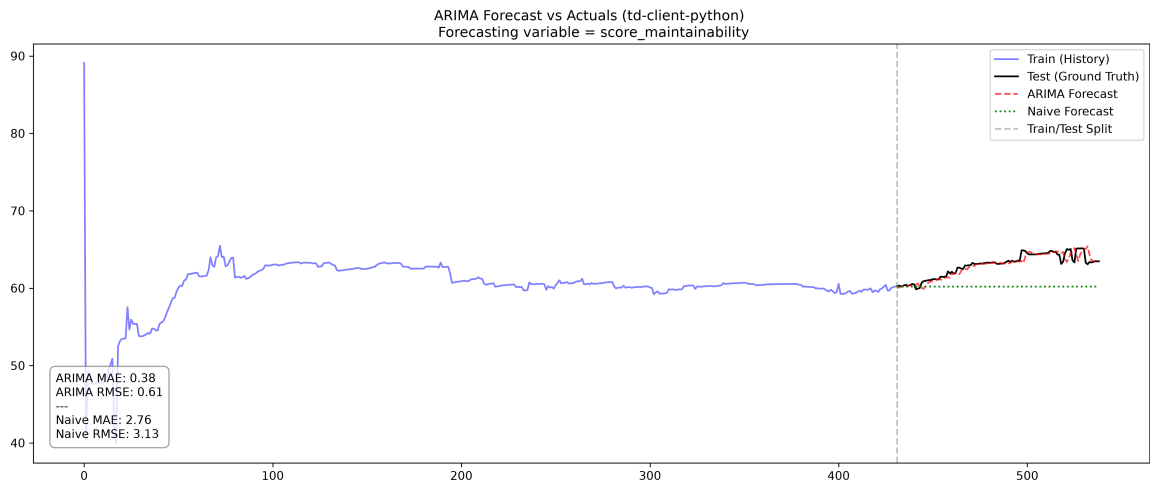
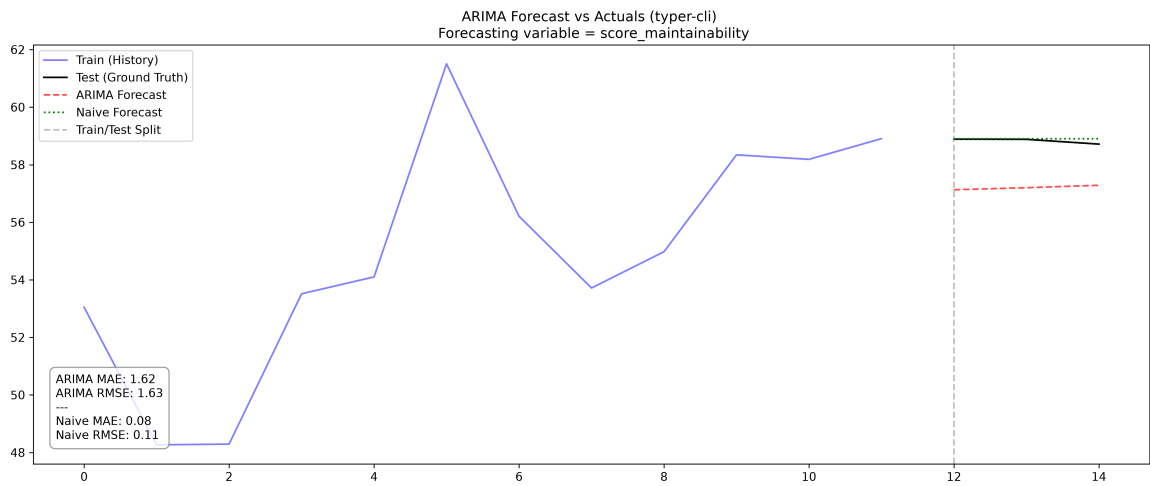


Figure B.6: Td-client-python ARIMA Maintainability forecast.



Typer-cli ARIMA Maintainability forecast.

References

- [1] *Human-Written vs. AI-Generated Code: A Large-Scale Study of Defects, Vulnerabilities, and Complexity*. 2025. arXiv: 2508.21634 [cs.SE]. URL: <https://arxiv.org/abs/2508.21634>.
- [2] *On Developers' Self-Declaration of AI-Generated Code: An Analysis of Practices*. 2025. arXiv: 2504.16485 [cs.SE]. URL: <https://arxiv.org/html/2504.16485v1>.
- [3] *The Squalle Model - A Practice-based Industrial Quality Model*. 2009. URL: <https://rmod-files.lille.inria.fr/Team/Texts/Papers/Mord09b-TechReport-Squalle.pdf>.
- [4] *Software quality metrics aggregation in industry*. URL: <https://cmustrudel.github.io/papers/smr1558.pdf>.
- [5] *A Perspective on Explainable Artificial Intelligence Methods: SHAP and LIME*. 2023. arXiv: 2305.02012 [cs.AI]. URL: <https://arxiv.org/html/2305.02012v3>.
- [6] Daniel Orel et al. "Droid: A Resource Suite for AI-Generated Code Detection". In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 2025.
- [7] *The Quamoco Product Quality Modelling and Assessment Approach*. 2012. URL: <https://teamscale.com/hubfs/26978363/Publications/2012-the-quamoco-product-quality-modelling-and-assessment-approach.pdf>.
- [8] *Fault Prediction based on Software Metrics and SonarQube Rules. Machine or Deep Learning?* 2021. arXiv: 2103.11321 [cs.SE]. URL: <https://arxiv.org/abs/2103.11321>.
- [9] *A software service supporting software quality forecasting*. URL: <https://upcommons.upc.edu/bitstreams/4d1b3444-87ae-4fb2-9434-9963feccf1e3/download>.

- [10] Junyoung Chung et al. *Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling*. 2014. arXiv: 1412.3555 [cs.NE]. URL: <https://arxiv.org/abs/1412.3555>.
- [11] Ailing Zeng et al. “Are Transformers Effective for Time Series Forecasting?” In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 37. 9. 2023, pp. 11121–11128. DOI: 10.1609/aaai.v37i9.26317. URL: <https://ojs.aaai.org/index.php/AAAI/article/view/26317>.
- [12] Peiwan Tang and Weitai Zhang. *Unlocking the Power of Patch: Patch-Based MLP for Long-Term Time Series Forecasting*. 2024. arXiv: 2405.13575 [cs.LG]. URL: <https://arxiv.org/abs/2405.13575>.
- [13] Yuqi Nie et al. “A Time Series is Worth 64 Words: Long-term Forecasting with Transformers”. In: *International Conference on Learning Representations*. 2023. URL: <https://arxiv.org/abs/2211.14730>.